The Use of Ontology in the Process of Designing Adaptive Software Systems

2022 IEEE 17th International Conference on Computer Sciences and Information Technologies (CSIT) | 979-8-3503-3431-9/22/$31.00 ©2022 IEEE | DOI: 10.1109/CSIT56902.2022.10000528

Dmytro Fedasyuk
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
dmytro.v.fedasyuk@lpnu.ua
ORCID: 0000-0003-3552-7454

Illia Lutsyk
Software Department
Lviv Polytechnic National University
Lviv, Ukraine
illia.i.lutsyk@lpnu.ua
ORCID: 0000-0001-7117-5332

***Abstract*— The analysis of approaches and methods for creating adaptive and self-adaptive software systems has been carried out. It has been determined that most approaches require reconfiguration to change the functionality of the software. An ontology of the adaptive software system has been formed, which allows to present the main components of the software without reference to the structure of the subject area. The architecture of an adaptive software system was designed, which combines the principles of component-oriented and client-server architecture. Such solution will allow the software to be dynamically adapted depending on the needs of the user. An analysis of the characteristics of the ontological model based on quality assessment metrics has been carried out.**

Keywords*— ontology, architecture of software systems, adaptive software systems, self-adaptive software systems, ontology evaluation metrics

I. INTRODUCTION

The increase in the complexity of software development requires the creation of new methods for designing software systems. Constant changes and refinements of software requirements create new difficulties associated with the integration of new functionality or changes to existing ones. Given the changing needs and requirements of users, software systems require constant improvement and refinement, which, in turn, necessitates repeated stages of design, development and testing. An effective way to solve the problem is to design complex systems based on the principles of adaptation and self-adaptation. These approaches make it possible to generalize information about the information system and change its appearance or behaviour depending on the defined adaptation rules. However, despite the overall effectiveness, most methods require system reconfiguration when adding new functionality, which, in turn, complicates the adaptation process.

Therefore, taking into account the specific problems of the development of complex software systems, it is important to improve the design process of adaptive software systems that provide the possibility of dynamic adaptation of software components to new user requirements.

II. ANALYSIS OF RECENT RESEARCH AND PUBLICATIONS

Design approaches based on adaptation and self-adaptation methods are among the most effective and are aimed at improving the development process of complex software with a complex architecture and changing user requirements [1, 2].

In a number of studies, general principles and methods of designing and implementing self-adaptive systems have been defined. In particular, in [3, 4] the authors highlighted the problems of designing these systems, namely: the problems of defining new unified processes of software adaptation and decentralization of system elements. Based on the taxonomy of approaches, the authors identified new directions for the development of the process of adapting software systems:

- contextual adaptation – activators of managed resources are integrated into the process of forming judgments;
- decentralization of adaptation logic – involves the use of a decentralized management system;
- proactive adaptation – software adaptation is based on prediction.

The concept of creating context-adaptive user interfaces for commercial vehicles was revealed in the work of L. Schölkopf, M. Wolf [5]. The authors note that the use of context-adaptive interfaces makes it possible to detect the most recurring events. Based on the received information, it becomes possible to build different variations of the user interface and scenarios of system use for the problem area analyzed by them, defining the main "working phases" of commercial transport drivers.

The use of modern methods for the design of an adaptive web interface, based on the microservice architecture of the software, is presented in the study by A. Buhay, V. Oliynyk [6]. In this paper, the authors presented a new model of the adaptive user interface, which, unlike standard approaches, allows you to create rules for modifying the graphical interface, taking into account user feedback and previous experience.

The main principles of using a component-oriented architecture in combination with adaptation policies for the implementation of a self-adaptive system are disclosed in the paper [7]. The study presents an approach that consists the automated generation of initial settings and states used by the adaptive system to identify the reconfiguration process.

Therefore, the goal of the research is to improve the design process of adaptive software systems, which will allow to form functional features and a graphical interface based on the ontology without the need for reconfiguration of the system.

III. DESIGN OF A CONCEPTUAL MODEL OF AN ADAPTIVE SOFTWARE SYSTEM

One of the problems of designing and developing adaptive systems is the effective display of domain elements and their representation in the development process. An effective way to represent information about a subject area is to design a conceptual ontological model of a software

979-8-3503-3431-9/22/$31.00 ©2022 IEEE

Authorized licensed use limited to: CINVESTAV. Downloaded on October 20,2025 at 22:15:22 UTC from IEEE Xplore. Restrictions apply.

system. This representation allows you to provide information in a structured way and establishes functional dependencies between concepts and attributes of domain objects [8]. In its standardized form, the ontological model consists of several components:

- concepts – reflect objects of the subject area;
- relations (properties of objects) – represent connections between concepts of the ontological model;
- functional attributes – fields/attributes that expand information about the specified object of the ontological model.

In a formalized form, the meta-ontology of an adaptive software system (Fig. 1) can be represented as a combination of three elements [9, 10]:

$$O_{meta} = \langle C_{meta} \;\; R_{meta} \;\; Prop_{meta} \rangle \qquad (1)$$

where C_{meta} is a set of entities of the adaptive software system subject area containing information about users and possible configurations of the software; R_{meta} is a set of relations between entities of subject area; $Prop_{meta}$ is a set of properties characterizing the entities (concepts) of the subject area.

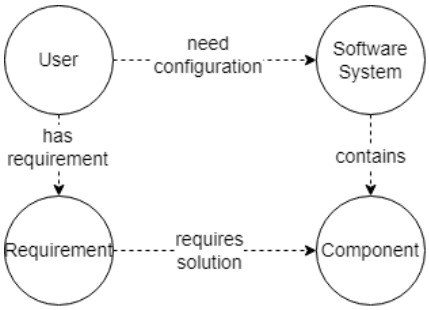

Fig. 1. Meta-ontology of an adaptive software system

The formed meta-ontology of adaptive software systems contains the main concepts and relationships necessary for defining the software configuration. This structure allows you to abstract from the specific software implementation for various subject areas. In addition, the generated model makes it possible to structurally expand the basic concepts by using the process of inheritance, as well as supplementing the model with new connections and rules. A detailed conceptual model of an adaptive software system is shown in Figure 2.

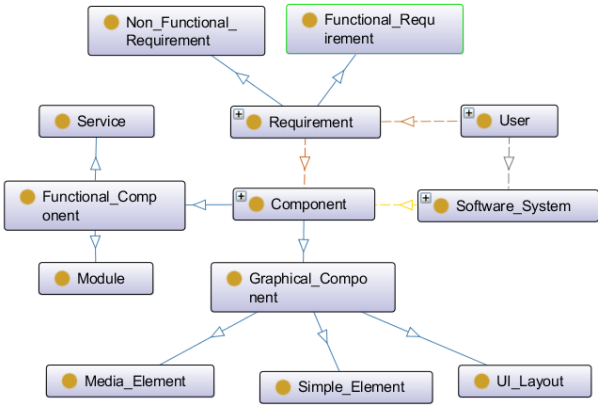

Fig. 2. Conceptual ontological model of an adaptive software system

In accordance with the created model, the user will receive the actual configuration of the software depending on changes in requirements or needs. Using the defined concepts, properties and relations, the SWRL rule for determining the settings of the software system was formed. SWRL is used to create a logical implication between the body of a rule and its conclusion consisting of individual atoms. In turn, atoms in the SWRL rule are represented as descriptions of entities and their properties, consisting of variables, instances or data.

For example, to determine the configuration of a software system based on user requirements and available software components, the following rule is formed:

$$\begin{aligned} &User(?user) \cap Requirement(?req) \\ &\cap \; Component(?comp) \cap \; Software\_System(?sys) \\ &\cap \; has\_requirement(user,?req) \\ &\cap \; requires\_solution(?req,?comp) \\ &\cap \; contains(?sys,?comp) \\ &\rightarrow need\_configuration(user,?sys) \end{aligned} \qquad (2)$$

where *?user, ?req, ?comp, ?sys* – SWRL-rule variables denoting, respectively, the entities *User, Requirement, Component* and *Software System*;

has_requirement, requires_solution, contains, need_configuration – the relationship between the entities of the subject area;

IV. Architecture of an Adaptive Software System

In addition to representing domain objects and relationships, another challenge in software system development is functionality change and modification. To ensure a certain level of adaptability, it is advisable to create a system using the principles of a three-level client-server and plug-in architecture.

The key feature of adaptive systems is a designed component-oriented architecture based on the use of plug-in [11]. Typically, the structure of a multi-module software system consists of a main component, software interfaces for connecting additional functional components, and actual additional modules. The ability to dynamically change the functional characteristics is provided through software interfaces that allow the exchange of information and interaction between modules, while removing the dependence on the specific implementation of additional system components.

This solution is effective because it eliminates the high coupling between the main and additional modules of the system. In addition, such a component-oriented architecture, due to the distributed structure, provides the ability to perform dynamic modification of both GUI elements and functional modules without reconfiguring the system. However, it should be noted that the design and use of the plug-in architecture requires the preliminary definition and implementation of additional structures and programming interfaces that provide a mechanism for connecting new modules or services.

However, using only a component-oriented architecture will not provide a sufficient level of abstraction, as it will require a separate implementation of the system configuration process for each device. In this case, it is advisable to use a combination of component-oriented and client-server architecture. To display the architecture of the

Authorized licensed use limited to: CINVESTAV. Downloaded on October 20,2025 at 22:15:22 UTC from IEEE Xplore. Restrictions apply.

software system proposed by us, the deployment diagram presented in Figure 3 is used.

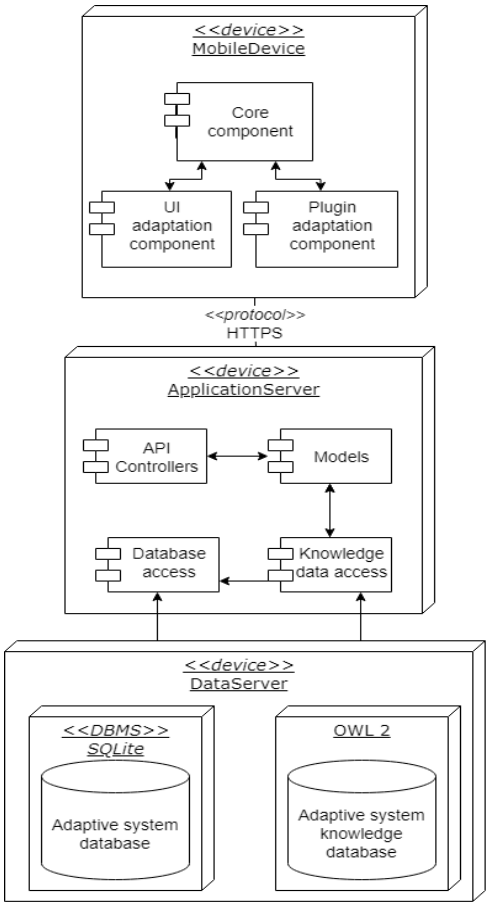

Fig. 3. The proposed architecture of the software system

The principle of designing software tools based on a three-level client-server architecture is to separate the system view from the levels of business logic and data [12]. This division makes it impossible to lose or damage data, in case a false or incorrect request is generated. In addition, such a solution allows you to replace or improve each level independently of each other.

The presentation level is presented in the form of a mobile application and is aimed at direct interaction with the user. To ensure extensibility, it is advisable to use the principles of plug-in architecture [11]. According to this approach, there are two types of components in the system:

- main – contains the main functionality and is actually the core of the software, to which additional functionality will be dynamically added at runtime;
- additional – contains additional functionality that complements or expands the basic component. A plug-in can be added and removed at any time without affecting the functionality of other plug-in.

This will ensure reusability, extensibility and interchangeability, as each module will be implemented independently of each other, and communication between modules will occur directly through the main component.

The business logic level is designed to process user data. Information coming from the presentation level is processed and sent to the data level, where the database and the ontological knowledge base are synchronized. Based on the information obtained from the data layer, personalized settings of the graphical interface and a list of necessary functional modules are formed.

The use of a three-tier client-server architecture will guarantee high reliability and security, scalability and configurability, as well as low performance requirements.

V. Assessment Metrics of the Ontology of the Adaptive Software System

Existing software quality models focus on the study of the characteristics of the software code, as well as on the processes associated with its development, modification, support and maintenance. At the same time, the presented quality assessments cannot be applied directly to ontological models, since they fundamentally differ not only structurally and functionally, but also have significant differences in the processes of design, development and support [13].

It is advisable to determine the quality of the developed ontological model using specialized metrics, which, in general, can be divided into two groups: hierarchical and relational [14]. To check the correctness of the structural and hierarchical characteristics of the ontology, it is advisable to use the following metrics that determine the depth, width, and complexity of the ontology, as well as cycle metrics.

Depth metrics. Determining the depth of the ontological model includes the calculation of the following indicators: absolute, average and maximum depth. The absolute depth is defined as the sum of lengths of all paths in the graph [13]:

$$A_{depth} = \sum_i^P l_i \ , \tag{3}$$

where P – the set of all paths of the ontological graph; l_i – length of the i-th path from the common set P.

The maximum depth of an ontological graph is defined as the maximum path from the set of all paths in the graph [13]:

$$Max_{depth} = l_j, \forall i \exists j (l_j > l_i) \ , \tag{4}$$

where l_i and l_j – path lengths j and i, respectively, from the set of paths of the ontological model.

Breadth metrics. Determining the breadth of the ontological model includes the calculation of the following indicators: absolute, average, maximum breadth. The absolute breadth is defined as the sum of the number of vertices at each level of the hierarchy [13]:

$$A_{breadth} = \sum_i^L N_i \ , \tag{5}$$

where L – the set of all levels of the ontological graph; N_i – the number of vertices at the i-th level of the graph.

The maximum breadth of the ontological graph shows the number of vertices on the level with the largest number of vertices and is determined by the following formula [13]:

$$Max_{breadth} = N_j, \forall i \exists j (N_j > N_i) \tag{6}$$

where N_j and N_i – number of vertices at levels j and i, respectively, from the set of levels of the ontological model.

Entanglement and loop metrics. For the group of entanglement indicators, 2 metrics are distinguished: the

Authorized licensed use limited to: CINVESTAV. Downloaded on October 20,2025 at 22:15:22 UTC from IEEE Xplore. Restrictions apply.

degree of entanglement and the average number of parent vertices for a graph vertex.

The degree of entanglement of ontology is determined by calculating the ratio of number of vertices that directly have several parent elements to the total number of vertices of the ontology graph. The smaller the total value, the better the ontology from the point of cognitive ergonomics [13]:

$$T_{ang} = t / n_g \quad (7)$$

where t – number of vertices with more than one parent (multiple connection is-a); n_g – total number of vertices in the ontological graph.

Cyclicity metrics are used to determine ontology quality. To determine the presence of cyclicity in the graph, we calculated: the number of different cycles in graph and degree of cyclicity. The degree of cyclicity of a graph is defined as the number of vertices included in any cycle in the total number of vertices of the ontological graph [13]:

$$A_{cycle} = N_{cycle} / n_g \quad (8)$$

where N_{cycle} – the number of vertices included in any cycle of the ontological graph;

An increase in the number of cycles in the ontological model may cause a delay in the development of ontological rules, since their evaluation is a resource-intensive process. The results of the assessment of the proposed model based on the specified metrics are presented in Table 1.

TABLE I. VALUES OF ONTOLOGY METRICS

Metric	Value	Metric	Value
Absolute depth	21	Maximum breadth	5
Maximum depth	3	Degree of entanglement	0
Absolute breadth	13	Degree of Cyclicity	0

The obtained values of the metric allow us to evaluate the quality of the ontological model based on the study of the structure of the hierarchy of entities and the relationships between them. Low indicators of depth and breadth allow you to work out ontological rules faster. The absence of cyclicity and multiple imitation indicates low resource consumption during processing of the formed semantic rules.

VI. CONCLUSIONS

The analysis of research on approaches to creating and maintaining self-adaptive systems confirmed the relevance of creating and improving methods of adapting software systems. Among the existing methods, the use of a conceptual model of the subject area is typical. It has been established that the main problem during such an adaptation process is the static structure of the corresponding model, which, in turn, doesn't allow making changes to the program or taking into account new configurations.

A conceptual model of an adaptive software system has been formed, which allows presenting the main elements and relationships without reference to the structure of the subject area. Defined concepts and relations allow you to form SWRL rules to determine the optimal system configuration depending on the software requirements.

The adaptive software architecture of the system was designed to facilitate the dynamic configuration of software. In combination with ontological model, such solution will allow adaptation taking into account both user requirements and the type of adaptation required.

The analysis of the characteristics of the ontological model for the adaptive software system based on specialized metrics was carried out. The absence of cyclicity and multiple imitation indicates low resource consumption during processing of the formed semantic rules.

REFERENCES

[1] C. Szabo, B. Sims, T. Mcatee, R. Lodge and R. Hunjet, "Self-Adaptive Software Systems in Contested and Resource-Constrained Environments: Overview and Challenges," in IEEE Access, vol. 9, pp. 10711-10728, 2021, doi: 10.1109/ACCESS.2020.3043440.

[2] K. Angelopoulos, A. V. Papadopoulos, V. E. S. Souza and J. Mylopoulos "Engineering self-adaptive software systems", ACM Transactions on Autonomous and Adaptive Systems, vol. 13 no. 1, 2018, pp. 1–27. doi: 10.1145/3105748.

[3] R. de Lemos et al., "Software Engineering for Self-Adaptive Systems: Research Challenges in the Provision of Assurances", Software Engineering for Self-Adaptive Systems III. Assurances, pp. 3-30, 2017. doi: 10.1007/978-3-319-74183-3_1.

[4] R. Calinescu, D. Weyns, S. Gerasimou, M. Iftikhar, I. Habli and T. Kelly, "Engineering Trustworthy Self-Adaptive Software with Dynamic Assurance Cases", IEEE Transactions on Software Engineering, vol. 44, no. 11, pp. 1039-1069, 2018. doi: 10.1109/tse.2017.2738640.

[5] L. Schölkopf, M. Wolf, V. Hutmann and F. Diermeyer, "Conception, Development and First Evaluation of a Context-Adaptive User Interface for Commercial Vehicles", 13th International Conference on Automotive User Interfaces and Interactive Vehicular Applications, 2021. doi: 10.1145/3473682.3480256.

[6] A. Buhay and V. Oliynyk, "Conceptual model of an adaptive web user interface using intelligent technologies", Adaptive automatic control systems, vol. 1, no. 32, pp. 15-22, 2018. doi: 10.20535/1560-8956.32.2018.145543.

[7] F. Dadeau, J. Gros and O. Kouchnarenko, "Automated Generation of Initial Configurations for Testing Component Systems", Formal Aspects of Component Software, pp. 134-152, 2021. doi: 10.1007/978-3-030-90636-8_8

[8] D. Fedasyuk and I. Lutsyk, "Tools for adaptation of a mobile application to the needs of users with cognitive impairments," 2021 IEEE 16th International Conference on Computer Sciences and Information Technologies (CSIT), 2021, pp. 321-324, doi: 10.1109/CSIT52700.2021.9648702.

[9] V. Lytvyn, Y. Burov, V. Vysotska, and V. Hryhorovych "Knowledge Novelty Assessment During the Automatic Development of Ontologies," in IEEE Third International Conference on Data Stream Mining & Processing (DSMP), 2020, pp. 372–377, doi: 10.1109/DSMP47368.2020.9204124.

[10] D. Fedasyuk and I. Lutsyk, "Method of modification of self-adaptive software systems based on ontology," 2022 IEEE 16th International Conference on Advanced Trends in Radioelectronics, Telecommunications and Computer Engineering (TCSET), 2022, pp. 530-533, doi: 10.1109/TCSET55632.2022.9766856.

[11] L. Sabatucci, V. Seidita and M. Cossentino, "The Four Types of Self-adaptive Systems: A Metamodel", Intelligent Interactive Multimedia Systems and Services 2017, pp. 440-450, 2017. doi: 10.1007/978-3-319-59480-4_44.

[12] H. Yokoyama, "Machine Learning System Architectural Pattern for Improving Operational Stability," 2019 IEEE International Conference on Software Architecture Companion (ICSA-C), 2019, pp. 267-274, doi: 10.1109/ICSA-C.2019.00055.

[13] H. Zhu, D. Liu, I. Bayley, A. Aldea, Y. Yang and Y. Chen, "Quality model and metrics of ontology for semantic descriptions of web services", Tsinghua Science and Technology, vol. 22, no. 3, pp. 254-272, 2017. doi: 10.23919/tst.2017.7914198.

[14] R. Lourdusamy and A. John, "A review on metrics for ontology evaluation," 2018 2nd International Conference on Inventive Systems and Control (ICISC), 2018, pp. 1415-1421, doi: 10.1109/ICISC.2018.8399041.

Authorized licensed use limited to: CINVESTAV. Downloaded on October 20,2025 at 22:15:22 UTC from IEEE Xplore. Restrictions apply.